

## TP4 : Virtualisation sous Docker - compose, build, publication

Le troisième cours Docker a **introduit** la notion de **composition** d'une stack de services docker dans un fichier yaml. Cette stack peut ensuite être **exécutée/publiée** en la "**buildant**" si nécessaire grâce à des commandes. Vous travaillerez en petit groupe de 2 ou 3 (si nombre impair d'étudiants).

Toutes les manipulations sont à faire en ligne de commandes.

### Application des notions du cours : web\_stackMP

Il vous est demandé de créer une stack docker contenant **un serveur web** (apache + php8.2 + nano + git + pdo pour mysql), un serveur de bdd **mysql en version 9.2** et **phpmyadmin**.

Pour ce faire, vous allez **créer** dans un dossier **web\_stackMP**, un fichier docker-compose.yml dans lequel on retrouvera, en plus des services un **network (network web\_stack de type bridge)**, un ou des **volumes** (à vous de trouver).

Les images de cette stack seront **build, publiées sur docker hub et harbinfo (s'il fonctionne) et versionné sur gitlab dans web\_stackMP**.

#### Travail à faire :

- créez dans votre ordinateur, un dossier web\_stack, ce dossier sera versionné et synchronisé avec un projet **gitlab** (url à communiquer ici : [https://docs.google.com/spreadsheets/d/13v2HyR9X8DRzTw6XmBUWLJ4zgk2EJdnnp\\_nfGZWkdny0/edit?usp=sharing](https://docs.google.com/spreadsheets/d/13v2HyR9X8DRzTw6XmBUWLJ4zgk2EJdnnp_nfGZWkdny0/edit?usp=sharing)).
- créez, à la racine de votre projet gitlab, une **documentation** en Markdown (fichier **readme.md**). Elle **présentera** une table des matières (que vous mettrez à jour au fur et à mesure), un tableau des différents services mis en place (nom du service, port externe, port interne, informations de connexion éventuelles, réseau, volume(s)) puis **expliquera** comment lancer vos stacks docker.
- créez, dans web\_stackMP, un fichier **docker-compose.yml** + un fichier **Dockerfile** ou un **Dockerfile inline** (au choix) contenant les services demandés, vous **choisirez** les **ports** et les **indiquerez** dans la **documentation**.
- Ajoutez** un fichier contenant vos **variables d'environnement** dans un fichier **séparé**

(choisissez le nom) et **modifiez** votre docker-compose.yml en conséquence. Testez.

- e. **ajoutez** dans la documentation les **commandes** permettant de taguer puis de publier les images de votre stack sur les **deux registry**. ⇒ 2\*3 images à push donc.

■ ex: sur **docker hub** ⇒ votrelogin/mysql:1.0, votrelogin/apache:1.0;  
votrelogin/phpmyadmin:1.0

■ ex: sur **harbinfo** ⇒ votrelogin/mysql:1.0, votrelogin/apache:1.0;  
votrelogin/phpmyadmin:1.0

- f. après vous être **connecté** sur le registry de **docker hub**, **pulez** votre image et **exécutez-la**. OK ? ⇒ pensez aux variables d'environnements.
- g. après vous être **connecté** sur le registry de l'iut **harbinfo**, **pulez** votre image et **exécutez-la**. OK ? ⇒ pensez aux variables d'environnements.

## Application des notions du cours : web\_stackPP avec postgres/pgadmin4

Il vous est demandé de créer une stack docker contenant **un serveur web** (apache + php8.2 + nano + git + pdo pour mysql), un serveur de bdd **postgres et pgadmin4 (comme vu au tp1)**.

Pour ce faire, vous allez **créer** dans un dossier **web\_stackPP**, vous allez créer un fichier docker-compose.yml dans lequel on retrouvera, en plus des services un **network (network web\_stack de type bridge)**, un ou des volumes (à vous de trouver).

Les images de cette stack seront **build, publiées sur docker hub et harbinfo (s'il fonctionne) et versionné sur gitlab dans web\_stackPP**.

A vous de jouer !

## Avec des données c'est mieux !

Dans le dossier **web\_stackMP** : travail avec Mysql / phpmyadmin

- **copier** les fichiers listeVoitures.php, ajoutVoituresGarage.php et creationBDD.php, ce dernier contiendra une instruction SQL du type create database...
- **faire** ce qu'il faut, en modifiant la partie **Dockerfile** (commande **COPY**), sans monter un volume, pour **copier** les fichiers listeVoitures.php, ajoutVoituresGarage.php dans /var/www/html et creationBDD.php dans le dossier /var/www/html/bd
- Testez maintenant vos scripts en vous rendant sur les bonnes url :

- [http://localhost/web\\_stackMP/creationBDD.php](http://localhost/web_stackMP/creationBDD.php)
- [http://localhost/web\\_stackMP/ajoutVoituresGarage.php](http://localhost/web_stackMP/ajoutVoituresGarage.php)
- [http://localhost/web\\_stackMP/listeVoitures.php](http://localhost/web_stackMP/listeVoitures.php)

Dans le dossier **web\_stackPP** : travail avec Postgres / pgadmin4

- **copier** les fichiers listeVoitures.php, ajoutVoituresGarage.php et creationBDD.php, ce dernier contiendra une instruction SQL du type create database...
- **faire** ce qu'il faut, en modifiant la partie **Dockerfile** (commande **COPY**), sans monter un volume, pour **copier** les fichiers listeVoitures.php, ajoutVoituresGarage.php dans /var/www/html et creationBDD.php dans le dossier /var/www/html/bd
- Testez maintenant vos scripts en vous rendant sur les bonnes url :
  - [http://localhost/web\\_stackPP/creationBDD.php](http://localhost/web_stackPP/creationBDD.php)
  - [http://localhost/web\\_stackPP/ajoutVoituresGarage.php](http://localhost/web_stackPP/ajoutVoituresGarage.php)
  - [http://localhost/web\\_stackPP/listeVoitures.php](http://localhost/web_stackPP/listeVoitures.php)

## Stack web avec serveur web js

Dans le dossier **web\_stackNJS**, créer une stack docker contenant **un serveur web** : nodejs, Vous choisirez les ports, noms etc.

La **documentation** se trouve ici : [https://hub.docker.com/\\_/node](https://hub.docker.com/_/node)

Faites en sorte **d'afficher** une page **d'accueil présentant** quelques **produits**, pour le code **JS** de la page, **utilisez une IA** !

Ensuite, créez trois pages web qui vont appeler en **AJAX** (toujours avec l'IA) les trois pages se trouvant sur :

- [http://localhost/web\\_stackNJS/creationBDD.php](http://localhost/web_stackNJS/creationBDD.php)
- [http://localhost/web\\_stackNJS/ajoutVoituresGarage.php](http://localhost/web_stackNJS/ajoutVoituresGarage.php)
- [http://localhost/web\\_stackNJS/listeVoitures.php](http://localhost/web_stackNJS/listeVoitures.php)

## Etude d'une stack web existante

En Programmation Web, les A2 du parcours RACDV travaillent depuis le début de l'année avec une image dont le dockerfile est le suivant :

```
# Use the official PHP image with Apache
FROM php:8.3-apache

# Set the ServerName to localhost
RUN echo "ServerName localhost" | tee -a /etc/apache2/apache2.conf
ENV TZ=Europe/Paris

# Enable Apache mod_rewrite
RUN a2enmod rewrite

# Install necessary PHP extensions
RUN apt-get update && \
    apt-get install -y \
        libpq-dev \
        tzdata \
        libsqlite3-dev \
        libaio-dev \
        unzip \
        nano \
        acl \
        wget && \
    docker-php-ext-install pdo pdo_mysql pdo_pgsql pdo_sqlite

# Install Composer
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer

# Install npm
RUN curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash && \
    export NVM_DIR="$HOME/.nvm" && \
    [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" && \
    [ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" && \
    nvm install --lts

RUN echo "date.timezone = Europe/Paris" > $PHP_INI_DIR/conf.d/timezone.ini

# Install Xdebug
RUN pecl install xdebug && \
    docker-php-ext-enable xdebug

# Configure Xdebug
RUN echo "zend_extension=xdebug.so" > /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini && \
    echo "xdebug.mode=debug" >> /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini && \
    echo "xdebug.start_with_request=trigger" >> /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini && \
    echo "xdebug.client_host=host.docker.internal" >> /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini && \
    echo "xdebug.client_port=9003" >> /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
```

```
# PHP Ini development
RUN cp $PHP_INI_DIR/php.ini-development $PHP_INI_DIR/conf.d/php.ini

# Set up the working directory
WORKDIR /var/www/html

# Copy the local project files to the container
# COPY . /var/www/html

# Expose port 80
EXPOSE 80

# Start Apache server
CMD ["apache2-foreground"]
```

1. Quelle commande permet de builder cette image ? Le build génère des erreurs (à corriger éventuellement)?

La commande qui permet de build est "docker compose build"

2. Quelle commande permet de run l'image build ?

La commande qui permet de build est "docker compose up -d"

3. Quelles sont les fonctionnalités installées ?

Dans ce dockerFile est installé un docker php en version 8.3 et apache. Il s'appellera localhost et sera sur la zone Europe/Paris pour l'heure. Le mode rewrite est enclenché.  
Les extensions php suivantes sont enregistrées: libpq-dev, tzdata, libsqlite3-dev, libaio-dev, unzip, nano, acl, wget.  
Ce qu'il faut pour les bases de données: pdo, pdo\_mysql, pdo\_pgsql, pdo\_sqlite.  
Composer est aussi installé, npm et Xdebug qui sert à debug du php.  
Le port exposé est choisi avec le port 80 et on travaille dans /var/www/html.

4. Est-il possible d'exécuter ce Dockerfile depuis un fichier docker-compose.yml ?

oui

## Https ?

- Trouver le moyen d'ajouter dans le dockerfile de la question précédente le support de https.
  - Quel code ajoutez-vous ?

```
RUN a2enmod ssl && a2ensite default-ssl  
EXPOSE 443
```

- Testez votre solution.
  - Donnez l'url ou les urls.

```
http://localhost:8080  
https://localhost:8443
```